

Name: 范振軒; B13303159

Position: backend

Designed the first few versions of backend logic:

(the final version was designed and implemented by others)

```
1  extends Node
2
3  signal item_inserted(value: int, index: int)
4  signal items_swapped(index_a: int, index_b: int)
5  signal item_removed(value: int)
6
7  @export var heap: Array[int] = []
8
9  func insert(value: int) -> void:
10     heap.append(value)
11     var index = heap.size() - 1
12
13     # draw the node first
14     item_inserted.emit(value, index)
15
16     # Then animate the sifting
17     _sift_up(index)
18
19  func pop_max() -> int:
20     if heap.is_empty():
21         push_error("Cannot pop from an empty heap!")
22         return -1
23
24     var max_val = heap[0]
25
26     # Handle heap with only 1 item
27     if heap.size() == 1:
28         heap.pop_back()
29         item_removed.emit(max_val)
30
31     return max_val
32
33     # animate the root swapping with the bottom node before deleting it
34     var last_index = heap.size() - 1
35     items_swapped.emit(0, last_index)
36
37     heap[0] = heap[last_index]
38     heap.pop_back()
39     item_removed.emit(max_val)
40
41     _sift_down(0)
42
43     return max_val
44
45  func peek() -> int:
46     if heap.is_empty():
47         return -1
48     return heap[0]
49
50  func heapify() -> void:
51     var start_index = (heap.size() / 2) - 1
52
53     for i in range(start_index, -1, -1):
54         _sift_down(i)
55
56  func _sift_up(index: int) -> void:
57     # Corrected Math
58     var parent_index = (index - 1) / 2
59
60     if index > 0 and heap[index] > heap[parent_index]:
```

```

60         var temp = heap[index]
61         heap[index] = heap[parent_index]
62         heap[parent_index] = temp
63
64         items_swapped.emit(index, parent_index)
65         _sift_up(parent_index)
66
67     func _sift_down(index: int) -> void:
68         var larger_index = index
69         var left_child = 2 * index + 1
70         var right_child = 2 * index + 2
71
72         # Ensure left child exists AND is larger
73         if left_child < heap.size() and heap[left_child] > heap[larger_index]:
74             larger_index = left_child
75
76         # Ensure right child exists AND is larger
77         if right_child < heap.size() and heap[right_child] > heap[larger_index]:
78             larger_index = right_child
79
80         # Base Case
81         if larger_index != index:
82             var temp = heap[index]
83             heap[index] = heap[larger_index]
84             heap[larger_index] = temp
85
86             items_swapped.emit(index, larger_index)
87             _sift_down(larger_index)

```

Implemented the first version of array visualization:

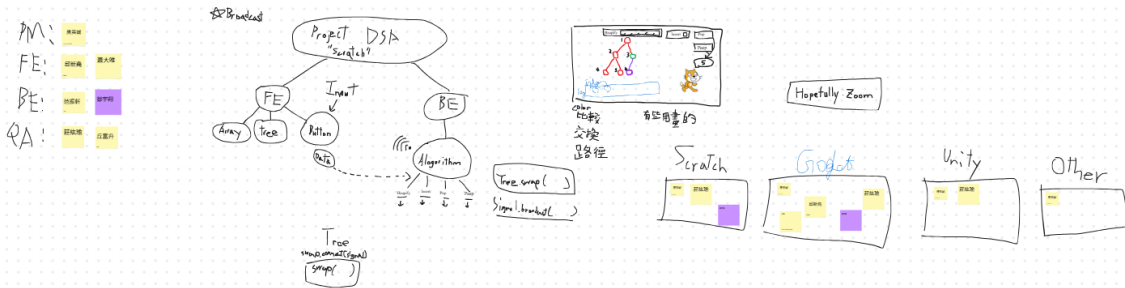
(the more detailed versions were modified by others)

```

1  class_name Heap_Array extends Node2D
2
3  var nodes: Array[TreeNode] = []
4  @onready var treeNode = preload("res://treeNode.tscn")
5
6  @export var horizontal_gap: float = 250.0
7  @export var start_x: float = -1000.0
8  @export var start_y: float = -350.0
9  @export var max_items_per_row: int = 20
10 @export var row_gap: float = 100.0
11
12 func rearrange_array() -> void:
13     for i in range(nodes.size()):
14         var col = i % max_items_per_row
15         var row = int(i / max_items_per_row)
16         var target_x = start_x + (col * horizontal_gap)
17         var target_y = start_y + (row * row_gap)
18
19         nodes[i].relocate(target_x, target_y)
20
21 func _ready() -> void:
22     nodes.clear()
23     SignalBus.item_inserted.connect(add_node)
24
25 func add_node(value: int, index: int) -> void:
26     var newNode: TreeNode = treeNode.instantiate()
27     newNode.set_value(value)
28     nodes.append(newNode)
29     add_child(newNode)
30     rearrange_array()
31

```

Discussion participation:



Timeline

Good learning 5/6 22:00

Checkpoint 5/9 22:00 (5/9, 5/10) 內卷時間

Finish 5/16 27:59

Video 5/17 20:00

Deadline 5/18 23:59